

BÁO CÁO BÀI TẬP LỚN – LEVEL 1: TIC-TAC-TOE MỞ RỘNG

Nguyễn Quốc Hiếu

26 April 2026

1 Overview

- **Họ và tên:** Nguyễn Quốc Hiếu
- **MSSV:** 25022853
- **Danh sách các nhóm chức năng đã cài đặt thành công:** Đã hoàn thiện toàn bộ 5 module theo yêu cầu của hệ thống: Game Engine, Game Logic, Game Interaction, Game Renderer và Bot Module. Chương trình có khả năng xử lý bàn cờ động kích thước $N \times N$ (từ 3×3 đến 12×12), hỗ trợ thay đổi điều kiện thắng (goal), và vận hành trơn tru cả 3 chế độ chơi (PvP, PvE, EvE).
- **Các cấp độ Bot đã triển khai:** Đã cài đặt đầy đủ 3 cấp độ Bot: **EASY** (Bắt buộc), **MEDIUM** (Tùy chọn - Heuristic) và **HARD** (Mở rộng - Minimax kết hợp Alpha-Beta Pruning).

2 Implementation Detail

2.1 Game Interaction

- **Xác thực dữ liệu (Input Validation):** Để đảm bảo chương trình không bị treo (crash) do dữ liệu đầu vào, quá trình nhập liệu được thực hiện qua hai bước chặt chẽ:
 - Trong hàm `validateInput`, chuỗi đầu vào được duyệt qua từng ký tự bằng vòng lặp và kiểm tra bằng hàm `std::isdigit()`. Nếu phát hiện ký tự không phải là số hoặc để trống, hàm lập tức trả về `false`.
 - Trong hàm `getInput`, chuỗi hợp lệ sẽ được chuyển đổi thành số nguyên thông qua `std::stoi()`. Đặc biệt, lệnh này được bọc trong một khối `try...catch(...)` để chủ động bắt lỗi tràn bộ nhớ (Overflow) khi người dùng cố tình nhập một số quá lớn.

- **Phản hồi của hệ thống:** Khi `getInput` hoặc các hàm xác thực phạm vi trả về `false`, hệ thống không vắng lỗi mà Game Engine sẽ in ra thông báo (ví dụ: “*nhập lại size đi b*”) và hiển thị lại menu để yêu cầu người dùng nhập lại.

2.2 Game Renderer

- **Định dạng hiển thị bàn chơi:** Bàn cờ được hiển thị trực quan dưới dạng một lưới vuông bằng các ký tự ASCII.
 - Hệ thống in ra một hàng ngang trên cùng đánh số thứ tự cột và một cột dọc bên trái đánh số thứ tự hàng (từ 0 đến `size - 1`). Điều này giúp người chơi dễ dàng định vị tọa độ.
 - Các vách ngăn được vẽ bằng ký tự `+---+` và `|`, phân tách các ô một cách rõ ràng.
 - Các ô chưa được đánh hiển thị bằng dấu `-`, các ô đã có nước đi sẽ hiển thị rõ ký hiệu X hoặc O.

2.3 Game Logic

- **Mã giả cho thuật toán checkWin:** Thuật toán duyệt qua toàn bộ lưới, quét theo 4 hướng (ngang, dọc, chéo chính, chéo phụ).

```
Hàm checkWin(board, size, symbol, goal, rule):
    Duyệt từng ô (i, j) trên bàn cờ:
        Nếu board[i][j] KHÁC symbol -> Bỏ qua
        Với mỗi Hướng (dx, dy) trong 4 hướng:
            Đếm số quân liên tiếp cùng symbol.
            Nếu (Đếm == goal):
                Tính tọa độ ô đầu mút (head) và đuôi (tail).
                Kiểm tra trạng thái trống của 2 mút.
                Nếu rule == OPEN_TWO:
                    Nếu cả 2 mút đều hỏ-> Trả về TRUE
    Trả về FALSE
```

Cách tối ưu hiệu suất: Dòng lệnh `if (board[i][j] != symbol) continue;` giúp thuật toán lập tức bỏ qua các ô trống hoặc ô của đối thủ, giảm thiểu đáng kể số lần thực hiện các vòng lặp đếm vô ích khi bàn cờ có kích thước lớn.

- **Logic hàm checkDraw:** Một trận đấu hòa xảy ra khi bàn cờ đã đầy nhưng không ai đạt điều kiện thắng. Hàm sử dụng vòng lặp kép duyệt mảng hai chiều. Nếu phát hiện bất kỳ ô nào còn chứa ký tự `-`, hàm trả về `false` (có thể chơi tiếp). Nếu duyệt hết mảng mà không có ô trống, trả về `true`.

2.4 Game Engine

- **Điều phối lượt đi:** Trong hàm `playGame()`, sử dụng vòng lặp cốt lõi `while (!isGameOver)`. Biến `currentPlayer` thay đổi luân phiên giữa 0 và 1. Dựa trên `GameMode` (PvP, PvE, EvE) và người chơi hiện tại, hệ thống đặt cờ `isBotTurn` để quyết định gọi hàm `botMove` hay yêu cầu người chơi nhập lệnh `getPlayerMove`.
- **Nhận biết kết thúc trò chơi:** Sau khi một nước đi được ghi nhận bằng `makeMove`, Engine lập tức gọi hàm `checkWin()`. Nếu trả về `true`, cập nhật cờ `isGameOver = true` và lưu trữ kết quả. Nếu `checkWin()` sai, tiếp tục gọi `checkDraw()`. Nếu hòa, gán `winner = DRAW_RESULT` và cũng kết thúc vòng lặp

2.5 Sử dụng Game Helper

- **Sử dụng `measureExecutionTime`:** Để đo thời gian thực thi của Bot, lời gọi hàm `botMove` được đặt bên trong một hàm Lambda `[&]() { return botMove(...); }` và truyền vào `measureExecutionTime`.
- **Trích dẫn Log:** [DEBUG] - [2026-04-26 09:45:12] execution time of [botMove()] = 5.17e-05s
- **Nhận xét:** Qua thực nghiệm, Bot cấp độ EASY tốn rất ít thời gian (gần như tức thời) do chỉ cần chọn một ô ngẫu nhiên. Cấp độ MEDIUM chậm hơn một chút do phải quét mô phỏng chặn đối thủ. Riêng cấp độ HARD (Minimax) tốn thời gian lâu nhất do độ phức tạp của cây trò chơi tăng theo cấp số nhân, đặc biệt là ở những nước đi đầu tiên.

2.6 Bot Module

Thuật toán được phân chia theo 3 cấp độ rõ rệt:

- **EASY (Ngẫu nhiên):** Quét toàn bộ bàn cờ, lưu tọa độ các ô trống (-) vào một mảng tĩnh. Sử dụng thư viện `<random>` để chọn ra một chỉ số ngẫu nhiên trong mảng và trả về tọa độ đó.
- **MEDIUM (Heuristic tham lam):** Áp dụng chiến lược nhìn trước 1 bước.
 - **Ưu tiên 1:** Đặt giả định quân của Bot vào các ô trống. Nếu `checkWin` báo thắng → Đánh để chiến thắng ngay.
 - **Ưu tiên 2:** Đặt giả định quân của Người chơi vào các ô trống. Nếu Người chơi thắng → Đánh vào đó để chặn thế cờ.
 - **Ưu tiên 3:** Chiếm ô trung tâm bàn cờ nếu còn trống. Nếu không, trả về hàm chọn ngẫu nhiên.
- **HARD (Minimax & Alpha-Beta Pruning):**

- Triển khai thuật toán Đệ quy Minimax để mô phỏng tương lai. Bot đóng vai trò Maximizing (tìm nước có điểm cao nhất), còn mô phỏng Người chơi là Minimizing.
- Tích hợp kỹ thuật cắt tỉa Alpha-Beta ($\beta \leq \alpha \rightarrow \text{prune} = \text{true}$) giúp bẻ gãy các nhánh tìm kiếm bất lợi, tăng tốc độ tính toán.
- Tối ưu hóa "Neighbor Search": Thay vì duyệt toàn bộ bàn cờ, Bot chỉ đánh thử vào các ô trống nằm kề cận các quân cờ đã có sẵn, giúp chống tràn bộ nhớ trên bàn cờ cỡ lớn.
- Hàm `lambda evalute` giúp tính toán thế cờ khi đạt MAX_DEPTH với quy tắc ô càng gần trung tâm thì càng lợi thế.
- Nếu ban đầu bàn cờ trống $\rightarrow$ ưu tiên đánh vào trung tâm.

3 Experiment Result

3.1 Giao diện tương tác

```
Tic-tac-toe Game (Version: 0.4.25022855)
-----
[INFO] - [2026-04-26 09:02:56] GameLogger initialized!
[INFO] - [2026-04-26 09:02:56] GameInteraction initialized!
>----- Tic-tac-toe [Console v0.4.25022855] -----<

Chọn kích thước bàn cờ đi bạn: (3<=size<=12)
3
Chọn số ô để thắng (3-5, goal<=size)
3
(1) PVP |(2) PVE |(3) EVE
3
Setup bot(X)
3
Setup bot(O)
1
[INFO] - [2026-04-26 09:03:03] GameEngine initialized!
```

Figure 1: Menu thiết lập

```
0 1 2
-----
#|X|O|-|
-----
1|-|X|-|
-----
2|O|-|-|
-----
Đang tính toán hàng triệu nước đi tiếp theo
[DBG] - [2026-04-26 09:03:09] execution time of [botMove()] = 5.16e-05s
0 (1,2) đã được kiểm soát[INFO] - [2026-04-26 09:03:10] GameEngine playing done!

0 1 2
-----
#|X|O|-|
-----
1|-|X|-|
-----
2|O|-|-|X|
-----
Bao ngày từ đầu là thua rồi mà
[DBG] - [2026-04-26 09:03:10] bạn có bot thua 01 thủ số 1 thắng (Tổng số nước đi: 5)
[INFO] - [2026-04-26 09:03:10] GameEngine show engine done!
[DBG] - [2026-04-26 09:03:10] GameInteraction closed!
[DBG] - [2026-04-26 09:03:10] GameLogger closing...
PS C:\Users\admin\Downloads\level-1 (1)\level-1>
```

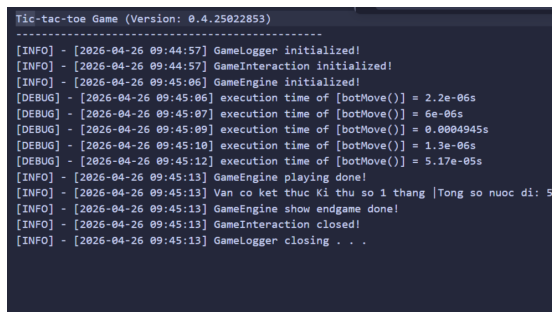
Figure 2: Bàn cờ và thông báo nước đi

3.2 Báo cáo grade

```
PS C:\Users\admin\Downloads\level-1 (1)\level-1> python grader.py --target_game
Grading start for game...
-----
[1/5] Testing: input1.txt... PASSED
[2/5] Testing: input2.txt... PASSED
[3/5] Testing: input3.txt... PASSED
[4/5] Testing: input4.txt... PASSED
[5/5] Testing: input5.txt... PASSED
-----
GRADING OVERALL:
- Total testcase: 5
- Corrects: 5
- Accuracy: 100.00%
Good job! You've passed all testcase.
PS C:\Users\admin\Downloads\level-1 (1)\level-1>
```

Figure 3: Kết quả testcase

3.3 Trích dẫn Log



```
Tic-tac-toe Game (Version: 0.4.25022853)
-----
[INFO] - [2026-04-26 09:44:57] GameLogger initialized!
[INFO] - [2026-04-26 09:44:57] GameInteraction initialized!
[INFO] - [2026-04-26 09:45:06] GameEngine initialized!
[DEBUG] - [2026-04-26 09:45:06] execution time of [botMove()] = 2.2e-06s
[DEBUG] - [2026-04-26 09:45:07] execution time of [botMove()] = 6e-06s
[DEBUG] - [2026-04-26 09:45:09] execution time of [botMove()] = 0.0004945s
[DEBUG] - [2026-04-26 09:45:10] execution time of [botMove()] = 1.3e-06s
[DEBUG] - [2026-04-26 09:45:12] execution time of [botMove()] = 5.17e-05s
[INFO] - [2026-04-26 09:45:13] GameEngine playing done!
[INFO] - [2026-04-26 09:45:13] Van co ket thuc Ki thu so 1 thang |Tong so nuoc di: 5
[INFO] - [2026-04-26 09:45:13] GameEngine show endgame done!
[INFO] - [2026-04-26 09:45:13] GameInteraction closed!
[INFO] - [2026-04-26 09:45:13] GameLogger closing . . .
```

Figure 4: Trích dẫn đoạn log

4 Nhận xét và tổng kết bài làm

- Về phong cách lập trình (Procedural Programming):

- **Điểm mạnh:** Cách tiếp cận lập trình thủ tục giúp dễ dàng hình dung luồng chạy của trò chơi từ trên xuống dưới (top-down). Việc chia nhỏ thành các hàm giúp mã nguồn sáng sủa và dễ bảo trì ở quy mô nhỏ.
- **Điểm yếu:** Khi trò chơi trở nên phức tạp hơn, việc phải truyền các biến trạng thái (bàn cờ, kích thước, luật chơi) qua lại giữa hàng loạt các hàm trở nên rất cồng kềnh. Về lâu dài, mô hình Lập trình Hướng đối tượng (OOP) sẽ phù hợp hơn để đóng gói dữ liệu và hành vi của từng thực thể.

- Về giao diện người dùng (Terminal Interface):

- **Thú vị:** Việc vẽ bàn cờ bằng các ký tự ASCII và làm chủ không gian console mang lại trải nghiệm lập trình rất cốt lõi. Nó rèn luyện tư duy tính toán vòng lặp và tọa độ cực kỳ tốt.
- **Khó khăn:** Hạn chế về mặt thị giác (không có màu sắc động, hoạt ảnh). Việc xử lý rác bộ đệm khi nhập liệu (`std::cin`) và liên tục xóa màn hình cũng đòi hỏi lập trình viên phải xử lý vô cùng cẩn thận để tránh chương trình bị treo.

- Về tổ chức mã nguồn (Monolithic file):

- Việc gộp toàn bộ mã nguồn vào một file duy nhất mang lại ưu điểm là dễ dàng biên dịch chỉ với 1 dòng lệnh.
- Tuy nhiên, khi code dài hơn 1000 dòng, việc cuộn chuột tìm kiếm hàm trở nên rất khó khăn. Để hệ thống chuyên nghiệp và dễ làm việc nhóm hơn, chương trình nên được tách thành các cặp file Header

(.h) và Source (.cpp) tương ứng với từng module riêng biệt (ví dụ: `GameLogic.h`, `GameLogic.cpp`).

- **Bài học rút ra:** Khó khăn lớn nhất mà em đã vượt qua là việc cài đặt thuật toán **CheckWin** và **đệ quy Minimax cho Bot mức độ HARD**. Thông qua dự án này, em không chỉ rèn luyện được tư duy gỡ lỗi (debugging) mà còn hiểu sâu sắc về cách tối ưu hóa hiệu năng, cắt tỉa không gian tìm kiếm trong Trí tuệ Nhân tạo.

5 Ghi chú sử dụng AI (AI Disclosure)

- **Công cụ sử dụng:** Google Gemini / Claude.
- **Mục đích sử dụng:**
 - Hỗ trợ debugging code lỗi.
 - Giải thích chi tiết nguyên lý hoạt động của thuật toán Minimax và cách tích hợp kỹ thuật cắt tỉa Alpha-Beta Pruning.
 - Hỗ trợ căn chỉnh, sửa lỗi định dạng khi chuyển văn bản sang mã $\text{\LaTeX}$ trên Overleaf.
- **Mức độ sử dụng:** Khoảng 20% - 30% cho việc tra cứu cú pháp và gỡ lỗi. Toàn bộ logic lõi, cấu trúc luồng trò chơi và việc tích hợp các hàm đều được tự nghiên cứu, viết code và tinh chỉnh để khớp với yêu cầu của đề bài.

6 Nguồn tham khảo (References)

1. Tài liệu hướng dẫn Bài tập lớn Level 1 - Học phần Phương pháp luận lập trình (UET).
2. Ý tưởng cài đặt giao diện bàn cờ và thuật toán CheckWin của kênh youtube BroCode: https://youtu.be/D3OowXMzAm8?si=pk07_b-8X5kwSjhu
3. Ý tưởng thuật toán Minimax kết hợp cắt tỉa Alpha-Beta từ kênh youtube Sebastian Lague: <https://youtu.be/1-hh51ncgDI?si=bmhSggPUKecC2WTs>